

Project AI Cluster

Goal

Create a unified LLM network to distribute RAM across a pool of old/broken devices to be able to run a bigger model across that pool

Initial Set up

- Samsung s21 ultra (Half screen working)
- MacBook pro Intel (Screen not working)
- Samsung note 9 (Screen not working)
- Wifi Linksys modem

Steps:

First connected the modem and gave it a name "AI cluster" to the network for 5G, to be able to connect all the devices together to the same network isolated and fast.

Node 1: MacBook Pro (Intel Core)

- **Power Management Configuration:** Overrode default macOS sleep behavior using native kernel energy management flags via terminal (`sudo pmset -a disablesleep 1`). This allows the laptop to run continuously as a headless, lid-closed server without entering a low-power state or dropping its network card.
- **Network Infrastructure:** Configured SSH (Secure Shell) under System Settings to allow headless command-line administration from the main workspace laptop.
- **Toolchain Injection:** Deployed core developer utilities including `git` , `python` , `clang` , `nodejs` , and the `uv` fast-pip toolchain wrapper.



Node 2: Galaxy S21 Ultra (Android 14 / One UI)

- **Environment Provisioning:** Configured a sandboxed Linux subsystem running natively on the ARM64 architecture using the latest stable F-Droid distribution of Termux.
- **Repository Alignment:** Successfully updated the system package manager mirrors to stable Grimler trees and performed a full core upgrade (`pkg update && pkg upgrade`).
- **Toolchain Injection:** Compiled and built core development dependencies natively for Android: `git` , `python` , `clang` , and `nodejs-lts` .
- **Execution Engine:** Installed the native Termux-compiled version of the `uv` dependency manager to bypass standard Linux binary compatibility errors on Android.

- **Framework Assembly:** Cloned the core `exo` repository and successfully executed the local `npm install` and `npm run build` steps to compile the visual Node.js dashboard interface natively on the phone. This node currently holds the deployment files for the rest of the array.



Node 3: Galaxy Note 9 (Android 10 / One UI Legacy)

- **Environment Provisioning:** Deployed the native F-Droid Termux subsystem optimized for legacy Android APIs.
- **Repository Alignment:** Rewired internal Termux repository maps using `termux-change-repo` to force connection via stable mirrors, bypassing broken default package indexing.
- **Toolchain Injection:** Deployed matching core dependencies: `git`, `python`, `clang`, and `nodejs-lts`.
- **Execution Engine:** Pulled the custom `uv` package natively from the Termux repository tree to circumvent platform compilation issues (`aarch64-linux-android`).
- **Kernel Wake Lock:** Activated a permanent hardware wake lock (`termux-wake-lock`) to instruct the Android system kernel to keep the CPU cores active and fully clocked even when the device display dims or shuts off.
- **Operating System Optimization:** Modified internal One UI application parameters by whitelisting Termux to run as **"Unrestricted"** under battery management, preventing Android's background execution engine from killing active server threads.



Dashboard Screen: Galaxy Tab S6 Lite

- **Role:** Dedicated visual tracking console.
- **Infrastructure Configuration:** Prepped to host the P2P cluster mesh monitor network layer. Configured to listen for standard local HTTP requests broadcast from the nodes on port `52415`.

History:

Once the Note 9 and the Macbook were ready to be connected to the AI cluster wifi, placed them away. And once I downloaded the `exo` repository on the s21 Ultra, I transferred to the others by the Termux Linux since everything is connected to the same network.



Air-Gapped Local AI Infrastructure Experiment (Exo P2P Mesh)

Objective:

Built a 100% offline, air-gapped decentralized AI inference cluster using everyday hardware to distribute and pool memory resources for running frontier-level LLMs (specifically testing the **Gemma 4 12B Unified** model released in June 2026), entirely bypassing reliance on cloud layers or external network handshakes.

Target Hardware Pool:

- **Primary Node:** MacBook Pro (16GB Unified Memory / Apple Silicon Architecture)
- **Secondary Mobile Edge Nodes:** Samsung Galaxy S21 Ultra & Galaxy Note 9 (Termux/POSIX Environments)

Technical Engineering Challenges & Troubleshooting:

- **Dependency Blockers (macOS):** Faced initial build crashes during setup due to `nodejs-wheel-binaries` throwing nested CMake build errors through development packages (`basedpyright`). Resolved by bypassing the standard development workflow and enforcing strict runtime-only synchronization (`uv sync --no-dev`).
- **Platform Lockfile Constraints (Android/Termux):** Encountered strict `uv.lock` validation errors on mobile devices due to non-standard platform tagging inside Termux. Bypassed the master repository lock constraints by isolating a clean `uv venv` sandbox and utilizing direct editable installations (`uv pip install -e .`).
- **Compilation & Hardware Bottlenecks at the Edge:** * *System Tooling Incompatibilities:* Hit native source-compilation failures on mobile when attempting to build `psutil` and the core Rust-based `exo-rs` cross-bindings, stemming from hardcoded Android kernel checks and interrupted cargo network fetch loops. Successfully bypassed non-critical metrics dependencies by surgically modifying project metadata definitions (`pyproject.toml`).
 - *Thermal & Power Realities:* The legacy Note 9 node triggered severe thermal panic limits, causing aggressive background process throttling and OS-level termination of Termux during intensive cryptographic and optimization tasks.

Final Architectural Conclusion:

The experiment successfully proved the power of local peer-to-peer P2P UI discovery and tensor mapping, but exposed the hard limitations of heterogenous edge-computing loops on resource-constrained mobile hardware.

The infrastructure was successfully re-architected into a **High-Performance Single-Node Server Model**:

1. **Computation Layer:** The Apple Silicon MacBook Pro handles the direct runtime engine and stores the compiled Hugging Face Safetensors natively via local memory allocations.
2. **Air-Gapped Execution:** Enabled absolute offline autonomy via the `EXO_OFFLINE=true` / `-no-downloads` runtime parameter, forcing the node to compute entirely from disk.
3. **Interface Matrix:** The Android devices were successfully offloaded from heavy matrix calculations and repurposed into wireless edge clients. They hook into the isolated

AI_Cluster_5G network mesh via local socket binds to interact with the high-speed backend model running on the Mac.